

Univerzitet u Beogradu
Matematički fakultet

Analiza projekta otvorenog koda

ChatterBot

Seminarski rad iz predmeta **Verifikacija softvera**

Autor: Bogdan Tomić
Broj indeksa: 1040/2025
GitHub: <https://github.com/BogBogdan>

Beograd, 2026.

Sadržaj

1	Uvod	2
2	O projektu koji sam analizirao	2
3	Okruženje	3
4	Alati	3
4.1	flake8	3
4.2	mypy	4
4.3	bandit	4
4.4	Jedinični testovi	4
4.5	Integracioni testovi	5
4.6	Property-based testiranje	5
5	Pronađeni problemi	5
5.1	Nalaz 1: parser datuma	6
5.2	Nalaz 2: deljenje nulom u Jaccard meri	7
5.3	Nalaz 3: zaštita od path traversal napada	7
5.4	Nalaz 4: mera sličnosti nije simetrična	7
5.5	Nalaz 5: semantička sličnost sa podrazumevanim modelom	8
5.6	Nalaz 6: lokalno vreme označeno kao UTC	8
5.7	Nalaz 7: povratna vrednost ne odgovara potpisu	8
5.8	Nalaz 8: menja se rečnik koji je pozivalac prosledio	9
5.9	Nalaz 9: bot ponavlja korisnikov ulaz	9
5.10	Nalaz 10: izvezena baza ne može da se uveze	9
6	Ograničenja	10
7	Zaključak	10
	Prilog: struktura repozitorijuma	10

1 Uvod

Tema ovog rada je analiza biblioteke ChatterBot pomoću šest alata za verifikaciju softvera. Kod biblioteke nisam menjao. Sve što sam pisao, dakle testovi, skripte i ovaj izveštaj, stoji u posebnom repozitorijumu, a sama biblioteka je u njemu git submodule zaključan na jedan komit.

Krenuo sam od alata koji samo čitaju kod, jer se brzo pokreću i odmah pokažu gde ima sumnjivih mesta. To su flake8, mypy i bandit. Posle toga sam pisao testove, prvo jedinične, pa integracione, pa property-based.

Našao sam deset problema. Jedan je bezbednosni i za njega sam napisao skriptu koja pokazuje da se ranjivost zaista može iskoristiti.

2 O projektu koji sam analizirao

ChatterBot je biblioteka za pravljenje chat botova. Ne koristi jezički model. Bot pamti rečenice u bazi, a kad dobije pitanje traži najbližnju zapamćenu rečenicu i vraća ono što je zapisano kao odgovor na nju.

Repozitorijum	https://github.com/gunthercox/ChatterBot
Grana	master
Heš komita	feafc81a00be8b8bb65622e4ddc89ebf7b3e1329
Verzija	1.2.13
Licenca	BSD-3-Clause
Veličina	oko 10.400 linija Python koda

Tabela 1: Podaci o analiziranom projektu

Kod je podeljen na sedam celina:

- `conversation.py` sadrži klasu `Statement`, koja predstavlja jednu rečenicu
- `storage/sql_storage.py` upisuje rečenice u SQLite bazu i čita ih nazad
- `tagging.py` i `components.py` preko spaCy prave indeks po kome se pretražuje
- `search.py` traži kandidate u bazi
- `comparisons.py` računa koliko su dva teksta slična
- adapteri u `logic/` biraju šta će bot odgovoriti
- `trainers.py` pune bazu podacima

Kada se pozove `bot.get_response("Hello")`, redosled je sledeći:

```

ulazni tekst
-> Statement objekat                chatterbot.py:202
-> preprocesori (ciscenje beline)    chatterbot.py:217
-> search_text preko spaCy tagera     chatterbot.py:226
-> logic adapteri daju predloge sa pouzdanoscu chatterbot.py:271
-> bira se predlog sa najvecom pouzdanoscu
-> ulaz i odgovor se upisuju u bazu  chatterbot.py:262
odgovor

```

Ovde je bitno jedno. Pretraga ne traži rečenicu koja liči na korisnikovo pitanje, nego rečenicu koja je nekad bila odgovor na nešto slično. Zato svaka rečenica u bazi ima dva indeksa, jedan za svoj tekst i jedan za tekst na koji odgovara. Bez toga se kasniji nalazi 9 i 10 teško razumeju.

3 Okruženje

Windows 11, Python 3.12.10, spaCy model `en_core_web_sm` 3.8.0. Verzije alata su `pytest` 9.0.3, `coverage` 7.14.0, `hypothesis` 6.145.1, `flake8` 7.3.0, `mypy` 1.19.0 i `bandit` 1.9.2. Sve verzije su zakucane u `requirements.txt`, da bi brojke iz izveštaja i dalje važile kad neko drugi pokrene alate.

Ponavljjanje rezultata:

```
git clone --recurse-submodules <url>
cd 2026_Analysis_ChatterBot

python -m venv .venv
.venv/Scripts/activate

bash scripts/priprema.sh
bash scripts/pokreni_sve.sh
```

4 Alati

#	Alat	Šta radi	Rezultat
1	flake8	provera stila i sitnih grešaka	3 prijave
2	mypy	provera tipova bez pokretanja koda	46 grešaka
3	bandit	traži poznate bezbednosne obrasce	6 nalaza
4	pytest, jedinični testovi	112 testova nad pojedinačnim funkcijama	svi prolaze
5	pytest, integracioni testovi	18 testova nad pravom bazom i spaCy modelom	svi prolaze
6	Hypothesis	sam pravi ulaze i traži kontraprimer	4 kontraprimera

Tabela 2: Alati korišćeni u radu

Testove sam pisao samo za jezgro, odnosno za put poruke od ulaza do odgovora. Statičke alate sam pustio nad celim paketom, jer ih to ništa ne košta, pa se u njihovim rezultatima vide i moduli koje nisam testirao.

4.1 flake8

Flake8 je linter. Kod ne pokreće nego ga čita i prijavljuje odstupanja od stila propisanog u PEP 8, uz mali skup pravih grešaka kao što je nekorisćena promenljiva. Većina

prijava je kozmetička, pa se prava greška u njima lako izgubi.

Pustio sam ga dva puta. Prvo sa konfiguracijom projekta, koja dozvoljava linije do 175 kolona, pa sa granicom od 100 kolona.

Sa projektnom konfiguracijom dobio sam tri prijave: predugačku liniju u `logic/llm_adapters.py`, nekorišćenu promenljivu u `storage/redis.py:564` i prazan red sa belinama u `trainers.py:421`. Sa strožom granicom ih ima 87, ali je 85 od toga samo dužina linije.

Jedina koja nešto znači je nekorišćena promenljiva. Rezultat poziva `add_texts` se dodeljuje pa nigde ne koristi, dok se sedamdesetak linija iznad isti takav rezultat koristi.

4.2 mypy

Mypy proverava tipove bez pokretanja koda, na osnovu anotacija u potpisima funkcija. Tako nalazi neslaganja koja bi se inače videla tek kada se ta grana izvrši, a možda i nikada. O bibliotekama bez opisa tipova ne zna ništa, pa deo njegovih prijava nije greška.

Prijavio je 46 grešaka. Veći deo je šum. Nedostaju opisi tipova za `yaml` i `dateutil`, ima i promenljivih kojima se u toku funkcije menja tip. Dve prijave su prave greške. Prva je `parsing.py:637`, gde se sabiraju `int` i `timedelta`. Druga se odnosi na `parsing.py:564` i `response_selection.py:50`, gde funkcije mogu da se izvrše do kraja i vrate `None` iako u potpisu piše drugi tip. Obe sam kasnije potvrdio testovima.

4.3 bandit

Bandit čita sintaksno stablo i traži poznate nesigurne obrasce, na primer raspakivanje arhive bez provere putanja. Svakom nalazu daje ozbiljnost i pouzdanost, ali kontekst u kome se kod koristi ne zna, pa svaki nalaz mora ručno da se proveri.

Prošao je kroz 7.674 linije i prijavio šest stvari, jednu visokog prioriteta, jednu srednjeg i četiri niskog.

Četiri niska se odnose na modul `random`, koji se koristi za nasumičan izbor odgovora, pa tu nema problema. Srednji je `requests.get` bez vremenskog ograničenja, zbog čega preuzimanje korpusa može da visi. Nalaz visokog prioriteta je opisan u nastavku.

Uz testove sam merio i pokrivenost koda alatom `coverage.py`. Pokrivenost pokazuje koji su iskazi bili izvršeni bar jednom dok su testovi radili, a ne i da li je kod ispravan. Koristio sam je kao spisak mesta koja testovi uopšte ne diraju, da bih znao gde još treba da pišem. Izveštaj se pravi zajedno sa testovima i može se pregledati u pregledaču, u fajlu `tools/unit-tests/results/htmlcov/index.html`, gde su izvršene linije zelene, a neizvršene crvene.

4.4 Jedinični testovi

Jedinični test proverava jednu funkciju odvojeno od svega ostalog, bez baze, bez mreže i bez diska. Ono što funkcija traži spolja zamenjuje se lažnim objektom koji ume samo ono što je toj funkciji potrebno. Zato se test izvršava u milisekundama, daje isti rezultat pri svakom pokretanju i, kada padne, pokazuje tačno koja funkcija ne radi.

Ovakvih testova ima 112, u šest fajlova, za preprocesore, mere sličnosti, `Statement`, izbor odgovora, `utils.py` i matematički adapter. To su moduli koji se mogu izolovati. Trenere, sloj za bazu i adaptore za jezičke modele ovaj sloj ne dodiruje, jer se oni bez svojih zavisnosti ne mogu ni pokrenuti.

Odavde su nalazi 5, 6, 7 i 8. Nalazi 2 i 4 su ovde dobili stalne testove, pošto ih je pre toga našao Hypothesis.

4.5 Integracioni testovi

Integracioni test proverava da li komponente rade zajedno, pa lažnih objekata nema. Koristi se prava SQLite baza na disku, pravi spaCy model i pravi treneri. Sporiji je od jediničnog i kada padne ne kaže odmah koja je komponenta kriva, ali je jedino mesto na kome se vide greške u spoju.

Da rezultat ne bi zavisio od okolnosti, svaki test dobija svoju praznu bazu u privremenom direktorijumu i briše je za sobom, a bot je podešen tako da pri pravljenju ne preuzima ništa sa interneta. Bez toga bi ishod zavisio od redosleda testova i od toga da li mreža radi.

Testova ima 18, u dva fajla. Jedan radi sa slojem baze, drugi sa celim tokom od treninga do odgovora. Ovde se izvršava ono što jedinični testovi ne dodiruju, pre svega `trainers.py` i `sql_storage.py`. Odavde su nalazi 9 i 10.

4.6 Property-based testiranje

Kod jediničnog testa ja biram ulaz i pišem očekivani izlaz, pa test vidi samo slučajeve kojih sam se setio. Kod property-based testa zapisujem svojstvo koje mora da važi za svaki ulaz, a alat sam pravi ulaze i traži onaj koji to svojstvo obara. Kada ga nađe, skraćuje ga dok ne dobije najmanji ulaz koji i dalje pada, pa je kontraprimer dovoljno kratak da se odmah upotrebi.

Zadavao sam četiri vrste svojstava: invarijante izlaza, idempotentnost preprocesora, refleksivnost i simetričnost mere sličnosti. Prve tri su prošle, simetričnost je pala i to je nalaz 4.

Nad parserom datuma nasumičan tekst nije dao ništa, jer ne liči na datum, pa sam ulaze sastavljao od reči koje parser prepoznaje. Na pet hiljada takvih fraza skripta `tools/hypothesis/nadji_kontraprimere.py` obori parser na četiri načina. Peti uzrok, 1a, traži da se u istom primeru poklope fraza sa `last month` i bazni datum u januaru, pa se u ovoj seriji ne pojavi. Podešeno je `derandomize=True`, pa neizmenjena skripta uvek daje isti spisak. Zato sam svih pet uzroka, opisanih u nalazu 1, prepisao u obične testove koji se izvršavaju uz ostale i ne zavise od toga šta generator izvuče.

5 Pronađeni problemi

#	Gde	Problem	Nađeno pomoću
1	<code>parsing.py</code>	parser datuma puca umesto da vrati prazan rezultat, pet nezavisnih uzroka	Hypothesis, mypy
2	<code>comparisons.py:183</code>	deljenje nulom za tekst od samih stop-reči	Hypothesis
3	<code>trainers.py:629</code>	zaštita od path traversal napada može da se zaobiđe	bandit

#	Gde	Problem	Nađeno pomoću
4	<code>comparisons.py:47</code>	mera sličnosti nije simetrična	Hypothesis
5	<code>constants.py:36</code>	semantička sličnost je neupotrebljiva sa podrazumevanim modelom	jedinični test
6	<code>conversation.py:92</code>	lokalno vreme je označeno kao UTC	jedinični test
7	<code>response_selection.py:50</code>	funkcija vraća <code>None</code> umesto obećanog tipa	mypy, jedinični test
8	<code>utils.py:26-27</code>	menja se rečnik koji je korisnik prosledio	jedinični test
9	<code>components.py, best_match.py:85</code>	bot ponavlja ono što je korisnik rekao	integracioni test
10	<code>trainers.py:75 i :310</code>	izvezena baza ne može da se uveze nazad	integracioni test

Tabela 3: Spisak pronađenih problema

Za svaki nalaz postoji test u `tests/` koji ga dokazuje i koji pada dok je greška u kodu. U nastavku je svaki opisan posebno: šta se dešava, gde je uzrok i šta bih promenio.

5.1 Nalaz 1: parser datuma

Funkcija `datetime_parsing` je javna i opisana je u dokumentaciji biblioteke. Kad u tekstu nema datuma treba da vrati praznu listu. Umesto toga baca izuzetak, iz pet nezavisnih razloga. Ulaz joj je korisnički tekst, pa to znači pad aplikacije.

#	Ulaz	Uzrok
1a	"last month" u januaru	<code>parsing.py:609</code> računa <code>datetime(godina, mesec-1, dan)</code> , pa u januaru dobije mesec 0
1b	"last month" 31. u mesecu	ista linija ne skraćuje dan na dužinu ciljnog meseca, a grana za "next month" to radi ispravno
1c	"end of the week"	<code>parsing.py:636</code> uzme <code>base_date.weekday()</code> , što je ceo broj, i sabere ga sa <code>timedelta</code>
1d	"29 hours"	izraz oblika N sati se tumači kao doba dana, pa se pravi datum sa satom 29
1e	"end of the monday"	funkcija na <code>parsing.py:564</code> nema granu za "end of the", vrati <code>None</code> , a pozivalac na to dodaje <code>timedelta</code>

Tabela 4: Pet uzroka pada parsera datuma

Uzroke 1c i 1e je mypy prijavio pre nego što sam ih uopšte izazvao, a Hypothesis je posle našao ulaze koji ih obaraju.

Svaki od pet uzroka ima svoj test. Bazni datum je u njima zadat, jer 1a i 1b zavise od toga kada se test pokreće.

5.2 Nalaz 2: deljenje nulom u Jaccard meri

JaccardSimilarity računa odnos veličine preseka i veličine unije skupova lema dva teksta, ali pre toga izbacuje stop-reči. Ako u tekstu nema ničega osim stop-reči, skup ostane prazan. Za dva takva teksta prazna je i unija, pa `comparisons.py:183` deli nulom:

```
numerator = len(statement_a_lemmas.intersection(statement_b_lemmas))
denominator = float(len(statement_a_lemmas.union(statement_b_lemmas)))
ratio = numerator / denominator
```

Nalaz je došao od Hypothesis-a, iz svojstva da rezultat mora biti između 0 i 1, a skraćeni kontraprimer je bio kratak tekst bez ijedne prave reči. Poređenje teksta 'the' sa samim sobom obara `ZeroDivisionError`, a isto važi i za praznu poruku i za rečenicu tipa 'is it?'. Pošto poređenje ne vraća broj nego baca izuzetak, pada ceo poziv `get_response`, a ne samo taj jedan kandidat. `Levenshtein` na istom ulazu vrati broj.

Ispravka je jedna grana. Prazna unija znači da nema šta da se poredi, pa treba vratiti 0.

5.3 Nalaz 3: zaštita od path traversal napada

Bandit prijavljuje `tarfile.extractall` u `trainers.py:640`. Projekat ima zaštitu, funkciju `safe_extract`, koja za svaki fajl iz arhive proverava da li ostaje u dozvoljenom direktorijumu:

```
def is_within_directory(directory, target):
    abs_directory = os.path.abspath(directory)
    abs_target = os.path.abspath(target)
    prefix = os.path.commonprefix([abs_directory, abs_target])
    return prefix == abs_directory
```

Funkcija `os.path.commonprefix` poredi stringove znak po znak, a ne putanje po delovima. Ako je dozvoljen `.../ubuntu_dialogs`, a fajl ide u `.../ubuntu_dialogs_evil/`, zajednički prefiks je opet `.../ubuntu_dialogs` i provera prolazi.

Skripta `tools/bandit/poc_path_traversal.py` pravi tar arhivu sa fajlom čija putanja izlazi iz dozvoljenog direktorijuma, pušta je kroz pravi `extract` iz biblioteke i proverava gde je fajl završio:

```
Rezultat extract()      : PROSAO bez izuzetka
Fajl van dozvoljenog dir.: True
ZAKLJUČAK: ZASTITA JE PROBIJENA
```

Arhiva se preuzima preko HTTP-a i bez provere kontrolne sume, pa napad nije samo teorijski. Rešenje je `os.path.commonpath` ili, od Python-a 3.12, `tar.extractall(path, filter='data')`.

5.4 Nalaz 4: mera sličnosti nije simetrična

Od mere sličnosti se očekuje da ne zavisi od redosleda argumenata. Kod `LevenshteinDistance` to ne važi: poređenje tekstova 'ab' i 'bacb' daje 0,67, a poređenje istih tih tekstova obrnutim redom daje 0,33.

Razlika je dvostruka, dakle nije reč o zaokruživanju. Uzrok je što `compare_text` zapravo ne računa `Levenshtein` rastojanje, koje jeste simetrično, nego `dif-`

`flib.SequenceMatcher(None, a, b).ratio()`. `SequenceMatcher` svoja dva argumenta ne tretira isto, jer se indeks podniski pravi nad drugim argumentom i poklapanja se traže u odnosu na njega.

Posledica je da rang kandidata zavisi od redosleda kojim su prosleđeni. Pretraga poredenje uvek poziva na isti način, pa se u integracionim testovima nije videlo da bot zbog ovoga bira pogrešno. Ako se već koristi `SequenceMatcher`, rezultat bi trebalo simetrizovati, na primer uzeti veću od dve vrednosti.

5.5 Nalaz 5: semantička sličnost sa podrazumevanim modelom

`SpacySimilarity` poredi tekstove pozivom `document_a.similarity( document_b)`, što podrazumeva da učitani model ima vektore reči. Podrazumevani model za engleski je u `constants.py:36` postavljen na `en_core_web_sm`, a taj model vektore nema. `spaCy` tada poziv ne odbija, nego se vraća na prosek unutrašnjih tenzora i uz to izda upozorenje.

Rezultat je da sve liči na sve. Rečenice `'the cat sat on the mat'` i `'a dog ran in the park'`, bez ijedne zajedničke sadržajne reči, dobiju sličnost preko 0,8, uz upozorenje `no word vectors loaded`. Kada su svi parovi ovako visoki, mera prestaje da razdvaja i izbor odgovora postaje nasumičan.

Ispravka je da se za jezike kod kojih se ova mera koristi mapira model sa vektorima, na primer `en_core_web_md`, ili da se poziv odbije sa jasnom greškom kada model vektore nema.

5.6 Nalaz 6: lokalno vreme označeno kao UTC

Klasa `Statement` u konstruktoru uzima trenutno vreme i, ako ono nema vremensku zonu, dodeljuje mu UTC:

```
self.created_at = kwargs.get('created_at', datetime.now())
...
# Set timezone to UTC if no timezone was provided
if not self.created_at.tzinfo:
    self.created_at = self.created_at.replace(tzinfo=timezone.utc)
```

`datetime.now()` bez argumenta vraća lokalno vreme, a `replace(tzinfo=...)` samo lepi oznaku i ne prevodi vrednost. Rečenica napravljena u Beogradu leti tako dobije vreme koje je dva sata ispred stvarnog UTC-a, ali stoji označeno kao UTC.

Test proverava obe polovine tvrdnje, dakle da oznaka jeste UTC i da je sama vrednost ipak lokalna.

Dok je bot na jednoj mašini, ovo se ne primećuje, jer su svi zapisi pomereni za isto. Problem nastaje kada se ista baza deli između mašina u različitim zonama ili kada se vremena porede sa nekim drugim izvorom, jer redosled poruka više nije tačan. Ispravka je `datetime.now(timezone.utc)`.

5.7 Nalaz 7: povratna vrednost ne odgovara potpisu

Potpis funkcije `get_most_frequent_response` obećava `Statement`. Funkcija broji pojavljivanja u petlji po listi kandidata, a promenljivu `matching_response` inicijalizuje na `None`. Za praznu listu kandidata petlja se ne izvrši nijednom, pa se vrati `None`.

Ovo je jedan od dva slučaja koja je `mypy` prijavio pre nego što sam napisao ijedan test za njih, a poziv sa praznom listom kandidata ga potvrđuje.

Pozivalac koji veruje potpisu će nad povratnom vrednošću pozvati `.text` i dobiti `AttributeError` daleko od mesta na kome je greška nastala.

Uz isti kod ide i jedna sitnija stvar. Poređenje je `count >= occurrence_count`, sa znakom veće ili jednako, pa kod izjednačenja pobeđuje poslednji kandidat u listi, a ne prvi. To nigde nije dokumentovano, pa sam i za to napisao test.

Ispravka je da potpis bude `Optional[Statement]` ako je `None` namerno, ili da funkcija nad praznom listom podigne izuzetak.

5.8 Nalaz 8: menja se rečnik koji je pozivalac prosledio

Adapteri se u ChatterBot-u mogu zadati rečnikom sa ključem `import_path`. Funkcija `initialize_class` taj rečnik dopunjuje i prosleđuje ga konstruktoru:

```
if isinstance(data, dict):
    import_path = data.get('import_path')
    data.update(kwargs)
    Class = import_module(import_path)

    return Class(*args, **data)
```

U tri linije su dva problema.

Prvo, `data` se prosleđuje kao `**kwargs`, a u njemu je i sam `import_path`. Klasa koja taj parametar ne prima puca sa `TypeError`, na primer `LowercaseTagger`.

Drugo, `data.update(kwargs)` menja rečnik koji je napravio pozivalac. Kada se napravi bot, u taj rečnik upadne cela konfiguracija bota, uključujući i listu `logic_adapters` u kojoj se nalazi baš taj isti rečnik, pa nastane ciklična struktura.

Posledica se vidi kada se ista konfiguracija upotrebi dvaput, jer drugi bot dobija rečnik zagađen podešavanjima prvog. Pokušaj da se takva konfiguracija zapiše u JSON ili prosto ispiše puca zbog ciklusa.

Ispravka je da se radi nad kopijom rečnika i da se `import_path` iz nje izbaci pre poziva konstruktora.

5.9 Nalaz 9: bot ponavlja korisnikov ulaz

Ako se bot istrenira sa `['Good morning', 'Good morning to you too']`, na pozdrav `'Good morning'` odgovori istim tim tekstom.

Obe rečenice dobiju isti indeks za pretragu, `ADJ:morning`, jer se stop-reči izbacuju. Adapter `BestMatch` u drugoj fazi traži rečenice sa istim indeksom kao pogodak, da bi imao više varijanti odgovora, pa u tu listu uđe i sam trenirani ulaz. Filter izbacuje samo ono što je izgovorio bot, prepoznato po `persona` koja počinje sa `bot:`, a rečenice unesene treningom takvu oznaku nemaju.

Sa parom `['Hello', 'Hi there!']` ovo se ne dešava, jer te dve rečenice dobiju različite indekse.

Nalaz drže dva testa. Prvi hvata simptom onako kako ga korisnik vidi, preko `get_response`, a drugi tvrdi da tager za obe rečenice vraća isti indeks.

5.10 Nalaz 10: izvezena baza ne može da se uveze

U opisu metode `export_for_training` piše da izvezeni fajl služi za treniranje drugih botova. Izvoz na `trainers.py:75` pravi ključ `conversations` i listu parova rečenica, a

`JsonFileTrainer` na `trainers.py:310` čita ključ `conversation`, u jednini, i očekuje listu rečnika. Dva kraja istog posla se ne slažu ni u imenu ključa ni u obliku podataka.

Greška se vidi tek kada se oba koraka urade jedan za drugim, pa to test i radi: izveze razgovor, proveri šta je tačno upisano u fajl, i onda taj isti fajl da `JsonFileTrainer-u`, koji na njemu puca sa `KeyError`. Fajl je čitljiv, pa podaci nisu izgubljeni, ali se ne može upotrebiti onako kako dokumentacija tvrdi.

Ispravka je da se ključ i oblik usklade na jednom mestu, ili, ako je zbog postojećih fajlova potrebno, da `JsonFileTrainer` prihvati oba oblika.

6 Ograničenja

Redis, MongoDB i Django slojeve nisam analizirao, jer prva dva traže pokrenut server, a treći Django projekat. Adaptere za jezičke modele nisam testirao sa pravim serverom. Mutaciono testiranje nisam radio. Primetio sam da ponovljeni trening gomila duplikate u bazi, ali to nisam merio. Nalaze nisam prijavio autoru projekta.

7 Zaključak

ChatterBot je uredno napisan projekat, ali rubni slučajevi nisu pokriveni. Tri prijave alata `flake8` na 10.400 linija i postojeći CI pokazuju da se projekat održava, a deset nalaza pokazuje dokle je to održavanje stiglo.

Ako se pet uzroka pada parsera broji odvojeno, sedam grešaka je u `parsing.py` i `trainers.py`. Ta dva modula nisu bez testova, čak naprotiv: `parsing.py` ih u samom projektu ima 53. Nijedan od tih 53 testova, međutim, ne očekuje izuzetak, i ni u jednom se ne pojavljuju fraze `last month` ni `end of the`. Pisani su da pokažu da parser radi kada mu se da ono što ume da obradi, a ne šta radi kada mu se da nešto drugo.

Broj testova sam po sebi ne govori ništa o tome da li je iko probao ulaz za koji se očekuje da ne prođe.

Alati se dopunjuju. Mypy je prijavio dve greške bez pokretanja koda, a Hypothesis je našao ulaze koji ih izazivaju. Sa druge strane, nalaze 9 i 10 ni jedan ni drugi ne bi našli, jer se vide tek kad se komponente spoje.

Prilog: struktura repozitorijuma

<code>ChatterBot/</code>	analizirani projekat (git submodule, komit <code>feafc81</code>)
<code>tests/</code>	
<code>unit/</code>	jedinicni testovi
<code>integration/</code>	integracioni testovi
<code>property/</code>	property-based testovi
<code>tools/</code>	po jedan folder za svaki alat: <code>run.sh</code> i <code>results/</code>
<code>unit-tests/</code>	<code>integration-tests/</code> <code>hypothesis/</code> <code>flake8/</code> <code>mypy/</code> <code>bandit/</code>
<code>scripts/</code>	
<code>priprema.sh</code>	instalira zavisnosti
<code>pokreni_sve.sh</code>	pokrece svih sest alata
<code>ProjectAnalysisReport.pdf</code>	ovaj dokument
<code>README.md</code>	osnovni podaci i uputstvo